

practice-coding/attention.py

```

1  import torch
2  import torch.nn as nn
3  import torch.nn.functional as F
4  import math
5
6
7  class RMSNorm(nn.Module):
8      def __init__(self, d_model, epsilon = 1e-8):
9          super().__init__()
10         self.eps = epsilon
11         self.d_model = d_model
12         self.gamma = nn.Parameter(torch.ones(d_model))
13     def forward(self, x):
14         return x / torch.sqrt(torch.mean(x**2, -1, keepdim=True) + self.eps) *
self.gamma
15
16
17  class LayerNorm(nn.Module):
18      def __init__(self, d_model, epsilon = 1e-8):
19          super().__init__()
20         self.eps = epsilon
21         self.d_model = d_model
22         self.gamma = nn.Parameter(torch.ones(d_model))
23         self.beta = nn.Parameter(torch.zeros(d_model))
24     def forward(self, x):
25         mean = x.mean(-1, keepdim = True)
26         std = x.std(-1, keepdim = True)
27         return self.gamma * (x - mean) / (std + self.eps) + self.beta
28
29
30  def testRmsNormAndLayerNorm():
31     rmsNorm = RMSNorm(d_model=2)
32     print("rmsNorm")
33     print(rmsNorm(torch.tensor([[1.0, 2.0],
34                                 [3.0, 4.0],
35                                 [5.0, 6.0]])))
36     layerNorm = LayerNorm(d_model = 2, epsilon = 1e-10)
37     print(layerNorm(torch.tensor([[1.0, 2.0],
38                                 [3.0, 4.0]])))
39
40
41  class MHA(nn.Module):
42      def __init__(self, d_model, num_heads, dropout=0.1):
43          super().__init__()
44         self.d_model = d_model
45         self.num_heads = num_heads
46         self.d_k = d_model // num_heads
47         self.q_proj = nn.Linear(d_model, d_model)
48         self.k_proj = nn.Linear(d_model, d_model)
49         self.v_proj = nn.Linear(d_model, d_model)
50         self.o_proj = nn.Linear(d_model, d_model)
51         self.dropout = nn.Dropout(dropout)

```

```

52
53     def forward(self, query, key, value, mask = None):
54         q = self.q_proj(query)
55         k = self.k_proj(key)
56         v = self.v_proj(value)
57         batch_size = query.size(0)
58         q = q.view(batch_size, -1, self.num_heads, self.d_k).transpose(1,2)
59         k = k.view(batch_size, -1, self.num_heads, self.d_k).transpose(1,2)
60         v = v.view(batch_size, -1, self.num_heads, self.d_k).transpose(1,2)
61         scores = torch.matmul(q, k.transpose(-2,-1))
62         scores = scores / torch.sqrt(torch.tensor(self.d_k).float())
63         if mask is not None:
64             scores = scores.masked_fill(mask == 0, float('-inf'))
65         scores = F.softmax(scores, dim = -1)
66         scores = self.dropout(scores)
67         output = torch.matmul(scores, v)
68         output = output.transpose(1,2).contiguous().view(batch_size, -1,
self.d_model)
69         return self.o_proj(output), scores
70
71
72     def testMHA():
73         batch_size, nums_head, seq_len, d_model = 32, 8, 10, 512
74         q = torch.randn(batch_size, seq_len, d_model)
75         k = torch.randn(batch_size, seq_len, d_model)
76         v = torch.randn(batch_size, seq_len, d_model)
77         mha = MHA(d_model = d_model, num_heads= nums_head)
78         mask = torch.ones(batch_size, 1, 1, seq_len)
79         output, attn_weights = mha(q, k, v, mask)
80         print("Output shape:", output.shape) # Should be (32, 10, 512)
81         print("Attention weights shape:", attn_weights.shape) # Should be (32, 8, 10,
10)
82
83
84     class SelfAttention(nn.Module):
85         def __init__(self, d_model = 2, row_dim = 0, col_dim = 1):
86             super(SelfAttention, self).__init__()
87             self.d_model = d_model
88             self.row_dim = row_dim
89             self.col_dim = col_dim
90             self.Wk = nn.Linear(in_features=d_model, out_features=d_model, bias=False)
91             self.Wq = nn.Linear(in_features=d_model, out_features=d_model, bias=False)
92             self.Wv = nn.Linear(in_features=d_model, out_features=d_model, bias=False)
93
94         def forward(self, hidden_size, mask = None):
95             k = self.Wk(hidden_size)
96             q = self.Wq(hidden_size)
97             v = self.Wv(hidden_size)
98             sims = torch.matmul(q, k.transpose(dim0 = self.row_dim, dim1 =
self.col_dim))
99             scaling = k.size(self.col_dim) ** -0.5
100             sims = sims * scaling
101             print("before")
102             print(sims)

```

```

103     print("before")
104     print("after")
105     if mask is not None:
106         sims = sims.masked_fill(mask = mask, value = float('-inf'))
107
108     print(sims)
109
110     print("after")
111
112     weights = F.softmax(sims, dim = self.col_dim)
113     attention_scores = torch.matmul(weights,v)
114     return attention_scores
115
116
117 def testSelfAttention():
118     ## create a matrix of token encodings...
119     encodings_matrix = torch.tensor([[1.16, 0.23],
120                                     [0.57, 1.36],
121                                     [4.41, -2.16]])
122
123     ## set the seed for the random number generator
124     torch.manual_seed(42)
125
126
127     mask = torch.tril(torch.ones(3, 3))
128     mask = mask == 0
129     print(mask)
130
131     ## create a basic self-attention object
132     selfAttention = SelfAttention(d_model=2,
133                                 row_dim=0,
134                                 col_dim=1)
135
136     ## calculate basic attention for the token encodings
137     print( selfAttention(encodings_matrix, mask))
138     print(selfAttention.Wq.weight.transpose(0, 1))
139     print(selfAttention.Wk.weight.transpose(0, 1))
140     print(selfAttention.Wv.weight.transpose(0, 1))
141
142
143 class BasicRotaryPositionEmbedding(nn.Module):
144     def __init__(self, dim, max_seq_len=512, base_freq=10000):
145         super().__init__()
146         self.dim = dim
147         self.max_seq_len = max_seq_len
148         self.base_freq = base_freq
149
150         # Create frequencies based on the embedding dimension
151         freqs = torch.arange(dim // 2).float()
152         self.inv_freq = self.base_freq ** (-freqs / (dim // 2))
153
154     def forward(self, x, seq_len):
155         # Create position tensor
156         position_ids = torch.arange(seq_len, device=x.device).unsqueeze(1)

```

```
157
158     # Calculate angles using inverse frequency
159     angles = position_ids * self.inv_freq.unsqueeze(0)
160
161     # Compute sin and cos embeddings
162     sin_emb = torch.sin(angles)
163     cos_emb = torch.cos(angles)
164
165     # Apply RoPE: Combine sin and cos embeddings to original embeddings
166     x1, x2 = x[..., ::2], x[..., 1::2] # Split into even and odd
167     x = torch.cat([-x2 * sin_emb, x1 * cos_emb], dim=-1)
168
169     return x
170
171
172 def testBasicRotaryPositionEmbedding():
173     # Example usage
174     batch_size = 2
175     seq_length = 10
176     embedding_dim = 16
177
178     # Initialize sample input tensor
179     sample_input = torch.randn(batch_size, seq_length, embedding_dim)
180
181     # Initialize the Rotary Position Embedding
182     rotary_embedding = BasicRotaryPositionEmbedding(dim=embedding_dim)
183
184     # Apply the rotary embedding
185     output = rotary_embedding(sample_input, seq_length)
186
187     print("Input shape:", sample_input.shape)
188     print(sample_input)
189     print(rotary_embedding)
190     print(output)
191     print("Output shape:", output.shape)
192
193
194
195 class positionEncoding(nn.Module):
196     def __init__(self, d_model, max_len = 10000, base_freq = 10000):
197         super().__init__()
198         pe = torch.zeros(max_len, d_model)
199         position = torch.arange(0, max_len, dtype = torch.float).unsqueeze(1)
200         div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-
math.log(base_freq) / d_model))
201         pe[:, 0::2] = torch.sin(position * div_term)
202         pe[:, 1::2] = torch.cos(position * div_term)
203         pe = pe.unsqueeze(0).transpose(0,1)
204         self.register_buffer('pe', pe)
205     def forward(self, x):
206         x = x + self.pe[:x.size(0), :]
207         return x
208
209 def positionEncoding():
```

```
210     d_model, max_len, batch_size = 512, 100, 32
211     pos_encoder = positionEncoding(d_model, max_len)
212     input_tensor = torch.zeros(batch_size, max_len, d_model)
213     output_tensor = pos_encoder(input_tensor)
214     print("Output shape:", output_tensor.shape)
215
216 class RotaryPositionEmbedding(torch.nn.Module):
217     def __init__(self, d_model):
218         super(RotaryPositionEmbedding, self).__init__()
219         self.d_model = d_model
220
221     def forward(self, x):
222         seq_len, batch_size, d_model = x.size()
223
224         # 构造旋转式的正弦余弦矩阵
225         position = torch.arange(seq_len, dtype=torch.float).unsqueeze(1)
226         index = torch.arange(0, d_model // 2, dtype=torch.float)
227         div_term = torch.exp(index * (-math.log(10000.0) / (d_model // 2)))
228
229         sinusoid_inp = position * div_term.unsqueeze(0)
230
231         # 计算sin和cos
232         sin, cos = torch.sin(sinusoid_inp), torch.cos(sinusoid_inp)
233
234         # 进行旋转操作
235         x1, x2 = x[..., 0::2], x[..., 1::2]
236         rotated_x1 = x1 * cos.unsqueeze(1) - x2 * sin.unsqueeze(1)
237         rotated_x2 = x1 * sin.unsqueeze(1) + x2 * cos.unsqueeze(1)
238
239         # 合并旋转结果
240         x = torch.stack([rotated_x1, rotated_x2], dim=-1).reshape(seq_len,
batch_size, d_model)
241         return x
242
243
244 def testRotaryPositionEmbedding():
245     d_model = 512
246     seq_len = 60
247     batch_size = 32
248
249     rope_encoder = RotaryPositionEmbedding(d_model)
250     input_tensor = torch.zeros(seq_len, batch_size, d_model)
251
252     output_tensor = rope_encoder(input_tensor)
253     print(output_tensor)
254
255
256 testRotaryPositionEmbedding()
257
258 # 这个句子有多少个token?
259 # Wk, Wq和Wv的权重矩阵的形状是什么? 初始化的数据是多少?
260
261 # 这些线性层的权重矩阵 (Wk, Wq, Wv) 的初始值是由 PyTorch 的线性层自动初始化的。
262 # PyTorch 使用了一种基于均匀分布的初始化方法, 具体来说, 权重通常是从一个范围为
```

```
263 | #  $[-1/\sqrt{\text{in\_features}}, 1/\sqrt{\text{in\_features}}]$ 的均匀分布中随机选择的。在这个代码中，由于  
    |  $\text{in\_features}=2$ ，因此权重矩阵的初始值范围为  
264 |  
265 |  
266 |  
267 |
```